In [2]:
```python
# S3Q1)
# a) Design a NumPy representation for a 30-day x 24-hour x 10-sensor dataset.
import numpy as np

dataset = np.random.rand(30, 24, 10)
print("Dataset shape:", dataset.shape)
# Axis 0 = day, axis 1 = hour, axis 2 = sensor.

# b) Suppose: 1000 observations x 20 sensors.
# Determine:
# - appropriate NumPy representation,
# - shape,
# - extraction of one sensor,
# - extraction of one observation,
# - and statistical operations.

data = np.random.rand(1000, 20)
print("\n\ndata:\n", data[0:5, 0:5])
print("\ndata shape:", data.shape)
print("data type:", data.dtype)
print("data size:", data.size)

# Extraction of one sensor (1000 observations)
sensor_1 = data[:, 0]
print("\nSensor 1 data shape:", sensor_1.shape)

# Extraction of one observation (20 sensors)
observation_1 = data[0, :]
print("Observation 1 data shape:", observation_1.shape)

# Statistical operations
print("\nMean:", round(np.mean(data), 2))
print("Standard deviation:", round(np.std(data), 2))
print("Mean of sensor 1:", round(np.mean(sensor_1), 2))
print("Standard deviation of sensor 1:", round(np.std(sensor_1), 2))

# c) Given a (10000, 50) matrix,
# Explain how to extract one feature and one observation efficiently.

# Extraction of one feature and one observation efficiently
matrix = np.random.rand(10000, 50)

# Extract one feature (column), e.g. feature 1
feature = matrix[:, 0]       # shape: (10000,)

# Extract one observation (row), e.g. observation 1
observation = matrix[0, :]   # shape: (50,)
```

```
Dataset shape: (30, 24, 10)


data:
 [[0.71040046 0.28832379 0.11586615 0.25562851 0.14825671]
 [0.82529114 0.43165075 0.89602368 0.08728451 0.10450979]
 [0.02551443 0.95277112 0.34596959 0.32833966 0.15630358]
 [0.65484528 0.08447581 0.27996015 0.99320136 0.7596545 ]
 [0.17083212 0.15547395 0.47211404 0.9199104  0.2845846 ]]

data shape: (1000, 20)
data type: float64
data size: 20000

Sensor 1 data shape: (1000,)
Observation 1 data shape: (20,)

Mean: 0.5
Standard deviation: 0.29
Mean of sensor 1: 0.49
Standard deviation of sensor 1: 0.28
```

In [ ]:

```python
# S3Q2) Create Using Series and Data Frame .
# Assume that you have 1000 students records of MSc Data Science
# a) Student Performance Analytics.
# Identify:highest performer, average marks,
#          students below threshold, attendance-risk students.
# b) Analyze a student-performance DataFrame.
# c) Build a DataFrame-based sales analysis.

# S3Q2a)
import pandas as pd
students = pd.DataFrame({
    "student_id": range(1, 1001),
    "age": np.random.randint(20, 35, 1000),
    "gender": np.random.choice(["M","F"], 1000),
    "marks": np.random.randint(0, 101, 1000),
    "attendance": np.random.randint(0, 101, 1000)
})

highest_performer = students.loc[students['marks'].idxmax()]
average_marks = students['marks'].mean()
students_below_threshold = students[students['marks']<50]
attendance_risk_students = students[students['attendance']<75]

print("Highest Performer:")
print(highest_performer)
print("\nAverage Marks:", round(average_marks, 2))
print("Number of Students Below Threshold:", len(students_below_threshold))
print("Number of Attendance Risk Students:", len(attendance_risk_students))
```

```
Highest Performer:
student_id     58
age            22
gender          M
marks         100
attendance     18
Name: 57, dtype: object

Average Marks: 48.98
Number of Students Below Threshold: 515
Number of Attendance Risk Students: 748
```

In [3]:
```python
# S3Q2b)
print("\nStudent DataFrame Analysis:")

print("\nShape of DataFrame:", students.shape)

print("\nFirst 5 Students:")
print(students.head())

print("\nSummary Statistics:")
print(students.describe())

# Average marks by gender
average_marks_gender = students.groupby("gender")["marks"].mean()

print("\nAverage Marks by Gender:")
print(average_marks_gender.round(2))

# Average attendance by gender
average_attendance_gender = students.groupby("gender")["attendance"].mean()

print("\nAverage Attendance by Gender:")
print(average_attendance_gender.round(2))

# Count of students by gender
gender_count = students["gender"].value_counts()

print("\nNumber of Students by Gender:")
print(gender_count)
```

```
Student DataFrame Analysis:

Shape of DataFrame: (1000, 5)

First 5 Students:
   student_id  age gender  marks  attendance
0           1   22      M     75          15
1           2   34      F     32          91
2           3   26      M     95          94
3           4   25      M     81          55
4           5   30      F     45          31

Summary Statistics:
        student_id          age        marks  attendance
count  1000.000000  1000.000000  1000.000000  1000.00000
mean    500.500000    26.861000    48.981000    49.76400
std     288.819436     4.127556    29.599946    29.08099
min       1.000000    20.000000     0.000000     0.00000
25%     250.750000    24.000000    23.000000    25.00000
50%     500.500000    27.000000    47.500000    51.00000
75%     750.250000    30.000000    74.250000    75.00000
max    1000.000000    34.000000   100.000000   100.00000

Average Marks by Gender:
gender
F    49.75
M    48.26
Name: marks, dtype: float64

Average Attendance by Gender:
gender
F    49.3
M    50.2
Name: attendance, dtype: float64

Number of Students by Gender:
gender
M    516
F    484
Name: count, dtype: int64
```

In [22]:

```python
# S3Q2c)

sales = pd.DataFrame({
    "product": ["Laptop", "Mouse", "Keyboard", "Monitor", "Laptop",
                "Mouse", "Monitor", "Keyboard", "Laptop", "Monitor"],
    "category": ["Electronics", "Accessories", "Accessories", "Electronics",
                 "Electronics", "Accessories", "Electronics", "Accessories",
                 "Electronics", "Electronics"],
    "quantity": [5, 20, 15, 8, 3, 25, 6, 18, 4, 10],
    "price": [60000, 800, 1500, 12000, 60000, 800, 12000, 1500, 60000, 12000]
})

# Calculate revenue
sales["revenue"] = sales["quantity"] * sales["price"]

print("\nSales Data:")
print(sales)

# Total revenue
```

```
total_revenue = sales["revenue"].sum()
print("\nTotal Revenue:", total_revenue)

# Total quantity sold by product
product_quantity = sales.groupby("product")["quantity"].sum()

print("\nQuantity Sold by Product:")
print(product_quantity)

# Revenue by product
product_revenue = sales.groupby("product")["revenue"].sum()

print("\nRevenue by Product:")
print(product_revenue)

# Revenue by category
category_revenue = sales.groupby("category")["revenue"].sum()

print("\nRevenue by Category:")
print(category_revenue)

# Best-selling product
best_selling_product = product_quantity.idxmax()

print("\nBest-Selling Product:", best_selling_product)
```

```
Sales Data:
    product     category  quantity  price  revenue
0    Laptop  Electronics         5  60000   300000
1     Mouse  Accessories        20    800    16000
2  Keyboard  Accessories        15   1500    22500
3   Monitor  Electronics         8  12000    96000
4    Laptop  Electronics         3  60000   180000
5     Mouse  Accessories        25    800    20000
6   Monitor  Electronics         6  12000    72000
7  Keyboard  Accessories        18   1500    27000
8    Laptop  Electronics         4  60000   240000
9   Monitor  Electronics        10  12000   120000

Total Revenue: 1093500

Quantity Sold by Product:
product
Keyboard    33
Laptop      12
Monitor     24
Mouse       45
Name: quantity, dtype: int64

Revenue by Product:
product
Keyboard     49500
Laptop      720000
Monitor     288000
Mouse        36000
Name: revenue, dtype: int64

Revenue by Category:
category
Accessories      85500
Electronics    1008000
Name: revenue, dtype: int64

Best-Selling Product: Mouse
```

In [ ]: